

9.5 Continue

For direct support in software development, the integration of AI functions directly into the development environment is the most convenient solution. There are numerous solutions for VS Code and the JetBrains integrated development environments (IDEs), of which GitHub Copilot is the best known.

There are also various alternatives with similar features, such as Bito, Cody, or Codeium. Most of these services offer a limited free version. However, if you use it on a daily basis and want to integrate the tool into your development process, you'll need the paid version in most cases. In addition, there is the problem that you need to send your data to a third-party provider.

But there's also another way! Both the *Continue* plug-in, which we present in this section, and the *Tabby* project ([Section 9.7](#)) implement a code wizard based on a locally executed LLM. This allows you to retain control over your data.

9.5.1 Installation

The Continue plug-in is available for VS Code and the JetBrains IDEs. After the installation, Continue connects to a language model of your choice, whereby local LLMs are also supported with Ollama. A quick start guide and further information on Continue can be found on the project's official website and in the Ollama blog:

- www.continue.dev
- <https://ollama.com/blog/continue-code-assistant>

The first step is to install the extension. Continue is currently available for VS Code and/or one of the JetBrains IDEs. When you open the extension for

the first time, you can choose between three different variants:

- **Free trial**

Here you log in with your GitHub account and receive 50 free requests. These are routed via a Continue proxy server and executed with their API keys. This variant is well suited for initial tests.

- **Local models**

This mode requires that you host your AI models locally and, for example, operate your own Ollama installation with associated models.

- **Best experience**

The third variant is a suggestion from the Continue team. The best commercial models are suggested here, but you'll need an API key for each of them.

To demonstrate the features of Continue, we use the VS Code extension in combination with a local installation of Ollama. In the default installation, Continue assumes the following models:

- **chat**

Initially, Continue uses Llama3 as the model for the chat feature. The documentation recommends a model variant with more than 30B parameters if possible, but this means that the system on which you're running Ollama must have the appropriate resources. As an alternative to Llama3, Continue recommends the open-source model DeepSeek Coder v2 or the commercial models Claude 3, GPT-4, and Gemini Pro.

- **autocomplete**

The StarCoder 2:3b model is the default setting for auto-completion. Continue recommends a model with a parameter number between 1B and 15B. Alternatives are the commercial model codestral and DeepSeek Coder v2 as an open-source version.

- **embeddings**

Continue uses nomic-embed-text for embeddings. Alternatively, you can use the commercial model, voyage-code-2.

All settings are saved in the extension's `config.json` file and can be changed at any time. Here, for example, you can configure the various models by defining the display name and specifying where the model comes from, for example, from Ollama, as well as the name of the model.

9.5.2 Usage

After activating Continue, the extension suggests extensions or code fragments when you enter code, which you can accept or ignore. You can use the key combination `Ctrl` + `I` or `cmd` + `I` to modify the code in the currently open file. Let's suppose you have a code section where a character string is returned that is currently in German but should be translated into English. Simply place the cursor in the corresponding line, press the key combination, and formulate the prompt, *Change this value to English*. Continue then suggests a translated character string that you only need to accept.

The Continue extension chat is a versatile tool. In the simplest case, you can use a prompt to ask any question, such as What are the parameters for `Array.splice`. The model behind Continue responds with a detailed explanation, which is displayed directly in the chat window. The chat also has two special functions:

- **Slash commands**

If you start the message with a slash (/), you'll receive a selection of pre-defined commands. You can use these commands, for example, to edit the selected code, add comments, or generate a commit message.

Continue also provides the option of creating custom commands. If you select the last item in the dropdown (**Build a custom prompt**), you'll receive a template for such a prompt. This feature is particularly useful for tasks you have to perform frequently.

- **Context**

You can use the @ sign to add additional context to the chat. This information helps to improve Continue's response by placing the prompt in the appropriate context.

The context feature of Continue makes the extension a useful tool in everyday work. The ability to ask questions about your own code in the chat and at the same time integrate external sources of information means that you can receive very specific answers. The built-in context providers include the following:

- **Code:** This context refers to specific functions or classes within your project.

- **Git Diff:** You can use the `@diff` context to refer to the changes in your current Git branch and have them explained.
- **Terminal:** The terminal provider refers to the content of the integrated command line of your development environment.
- **URL:** You can use this context to link to external websites. Continue loads the page, converts it to Markdown, and passes it to the model as context.
- **Jira Issues:** You can also refer to Jira issues and thus directly include the requirements of your project in your prompts or responses.

The quality of Continue's responses depends heavily on the models used and the available infrastructure. With its large commercial models, Continue is an ideal tool for everyday work. However, Continue also returns very good results in combination with Ollama, provided that sufficient storage space and computing power are available; otherwise, the responses are very slow, which makes the tool uninteresting for the development process. The decisive advantage of local execution is that your data doesn't leave your system, and you can integrate the Continue extension very deeply into your project using the context providers.